찾아라! 도깨비 — MVP 아키텍처

개발할 때 옆에 두고 보는 문서. 어떤 파일이 무엇을 하고, 무엇을 바꾸려면 어디를 고치는지.

1. 한눈에

엔진	Unity 6.3 LTS (6000.3.11f1), uGUI + TextMeshPro, Input System. 씬 1개 `Assets/Scenes/Game.unity`
씬 만들기	씬은 손으로 만들지 않고 `MvpSceneBuilder`가 코드로 전부 만들어요 (메뉴 Tools → 찾아라도깨비 → MVP 씬 만들기). 배치를 바꾸려면 이 파일을 고치고 다시 만들기
화면	패널 8개 (시작 · 대화 · 관찰 · 그리기 · 판정 · 결과 · 채점표 · 종료) 중 하나만 켜고, 그리기부터 채점표까지 위에 HUD. 화면 기준 1080×1899 = 팀 시안(2248×3952)을 줄인 크기, 시안 픽셀 좌표 그대로 배치
판정	멀티모달 LLM = Google Gemini (`gemini-3-flash-preview`, 무료 등급 : 한도는 AI Studio 에서 확인). 웹 · 에디터 모두 판정 중계 서버(Cloudflare Worker)를 거쳐요 (키는 서버에만). 프롬프트는 기획서 부록 B 문장 기준 (달라진 점은 `Docs/기획/5_기획_변경_기록.md`). 실패하면 간식 돌려주고 다시 제출
배포	웹 게임 = GitHub Pages (`gh-pages` 브랜치), 판정 서버 = `Server/judge` (wrangler), 코드 = `main` (보호 브랜치, PR 로 합침)

2. 폴더

```
FindDokkaebi/
  Assets/Scripts/     게임 코드 (아래 3장)
  Assets/Editor/      에디터 도구 : 씬 만들기 · 자동 테스트 · 판정 검사 · 웹 빌드 · 글꼴 굽기
  Assets/Data/        단계 데이터 Stage1~5 (.asset, 씬 만들 때 MvpSceneBuilder 가 채움)
  Assets/Sprites/     게임 그림 전부 : Background · Characters · Stage1~5(물건 원본 · 조합 그림) · UI(화면 부품). 파일별 연결은 Docs/개발/3_아트_구조.pdf
  (배경)              맨 뒤 Background 한 장 : 시작 BG_Start, 게임 중 BG_Room, 이야기 종료 BG_Picnic. 물건은 그 위에 따로 올려요
  Assets/Resources/Sound/   소리 파일. 이름이 맞으면 코드 없이 바로 나요 (Sfx)
  Assets/Fonts/       Jua(시안 글꼴, 구운 SDF) · Noto Sans KR(없는 기호 대신)
  ArtSource/          기획서 그림 · 첨부 사진 원본 (게임에서 안 씀)
  Server/judge/       판정 중계 서버 (worker.js · stages.json 게임 프롬프트 · appendix_b.json 부록 B 원문 · wrangler.toml)
  Docs/               읽는 문서 : 기획 · 전달 · 개발 · 기록 (안내는 0_문서_안내.md)
  Tools/docs/         문서를 만드는 스크립트, 받은 아트 ZIP 넣기(apply_art_zip.py), 그림 도구(art_pipeline)
  Builds/WebGL/       웹 빌드 결과 (git 에 안 올림)
  UserSettings/OpenAI.json   (선택) 판정 서버 주소가 비어 있을 때만 쓰는 OpenAI 키 — git 에 안 올림
```

3. 게임 코드 (Assets/Scripts)

파일	하는 일	주로 고칠 때
`GameManager`	화면 흐름(상태 머신) · 간식 · 타이머 · 판정 요청 · 결과 규칙 · 기록 · 일시정지	흐름 · 규칙
`GameScript`	게임의 모든 대사 (`Docs/기획/13_대사_트리.md`는 이 파일에서 자동으로 만들어요)	대사
`DialogueBox`	같은 크기의 대사 상자 : 얼굴, 한 글자씩 나오는 글자, 넘기기 화살표	대사 표시
`Sfx`	효과음 · 배경음. `Resources/Sound`에 파일이 없으면 조용히 넘어가요	소리 연결
`StageData`	단계 하나의 데이터 (이름 · 색 모드 · 원본 그림 · 특징 목록 · O·X 조합별 그림 `wrongSprites`). `Feature` = 특징 하나 (id · 이름 · 판정 기준 글 · 테스트용 계산 종류)	단계 · 특징 추가
`DrawingBoard`	512×604 쪽지 (클립보드의 하얀 면과 같은 비율) : 펜 · 지우개 · 굵기 · 되돌리기(20번) · 모두 지우기, 제출할 그림 텍스처. 그림판은 종이 면이 뚫린 틀 뒤에 있어서 잉크가 둥근 모서리 안에만 보이고, 그리다가 밖으로 나갔다 들어오면 쪽지 안에 걸친 부분만 그려요 (ClipToBoard)	그리기 느낌
`ToolBar`	색 · 연필 · 지우개 · 굵기 버튼, 고른 표시, 단계마다 색 버튼 보이기/숨기기	도구 버튼
`ColorMixer`	색 섞기 카드 : 물감을 누를 때마다 한 번씩 넣기 · 되돌리기 → 넣은 비율로 계산한 색 (Mix) → 섞은 색 칸	섞는 규칙
`OpenAIJudge`	AI 판정 요청 : 판정 서버 주소가 있으면 서버, 없으면 OpenAI.json 키로 OpenAI 직접. 프롬프트 (BuildPrompt) = 부록 B 문장 기준 (검은색 단계 = 방망이 프롬프트, 색 단계 = 돗자리 프롬프트, 단계 이름 · 목표 대상 · 특징만 바꿔 넣음) · 응답 JSON 해석(특징별 present)	프롬프트 · 서버 주소
`LocalJudge`	계산식 판정 (모양 · 색 계산). **자동 테스트 전용** — 실제 플레이에서는 안 씀	테스트
`IncenseBurn`	향 연출 : 줄기가 타고 불꽃 · 재 · 연기가 따라 움직임, 10초 전 강조. 그림은 `Tools/docs/art_pipeline/cut_incense.py`	향 움직임
`SimpleAnim`	코드로 만든 움직임 전부 : 등장 · 걷기 · 두리번 · 간식 · 물건 착지 · 채점 표시 · 꽃가루 (Kind 하나씩)	연출

화면 흐름 (GameManager.State)

```
Title —StartGame→ Dialogue(대사 → 쪽지) —StartObserve→ Observe(3초) —자동→ Draw(60초)
Draw —Submit(간식-1)→ Judge —결과, 찾는 동작 한 바퀴를 마친 뒤→ Result(도깨비 → 주인공)
Result —틀림 + 간식 남음→ 다시(Retry : 이전 그림에 이어, 60초) 또는 포기(ShowCompare)
Result —성공 또는 간식 없음→ Compare —NextStage→ Dialogue(다음 단계) … → Ending —터치 두 번→ Title
1단계를 못 찾으면 batLost : 이야기 종료까지 도깨비 그림이 빈손(dokkaebiNoBat)
Judge —AI 실패→ (간식+1) Result(오류 대사) —터치→ Draw(그리던 그림)
그리기 화면 —TogglePause→ 일시정지 (Time.timeScale 0)
```

결과 규칙 (ShowResult)

특징이 n개면 2^n가지. 전부 맞음 → 진짜 물건 A / 일부 → 그 조합의 A' / 전부 틀림 → 완전히 다른 물건 B (`StageData.WrongSpriteFor`). 실패했고 간식이 남으면「다시」·「포기」버튼이 대사 상자 자리에 나와요. 결과 화면에는 O · X 목록이 없고, 채점표에서만 보여 줘요.

4. 판정이 흐르는 길

```
제출 → board.Texture → PNG → base64
  ├ (판정 서버 주소 있음 = 보통)  판정 서버 POST /judge {stage, image}
  |     서버 : 허락한 곳(웹 게임 주소 · Unity)만, IP 당 1분 20번 → stages.json 의 그 단계 프롬프트 + 그림
  |           → Google Gemini (GEMINI_MODEL). 서버는 미국(aws:us-east-1)에서 돌아요 (홍콩에서 보내면 Gemini
가 거절)
  └ (서버 주소 없음)  UserSettings/OpenAI.json 키로 OpenAI Responses API 에 바로
← AI JSON {inputStatus, features:[{featureId, status, evidence}]} → present = (status == present) →
ShowResult
실패(연결 · 키 · JSON) → OnJudgeError : 간식 돌려주고 오류 대사 → 터치하면 그리던 그림으로
```

> 프롬프트를 바꾸면 : 특징 글은 `MvpSceneBuilder.CreateStageDataFiles`, 프롬프트 틀은 `OpenAIJudge.BuildPrompt` → 씬 만들기 → 판정 서버 프롬프트 내보내기(`Server/judge/stages.json`) → `Server/judge` 에서 `npx wrangler deploy`. 프롬프트는 서버에 있어서 웹 게임은 다시 빌드하지 않아도 돼요. 서버 키 : 키를 복사한 뒤 `Get-Clipboard | npx wrangler secret put GEMINI_API_KEY`. 무료 한도는 모델 · 프로젝트마다 달라요 (미국 태평양 시간 자정에 다시 채워짐 : 한국 시간 오후 5시, 서머타임 때는 오후 4시). 모델을 바꾸려면 `Server/judge/wrangler.toml` 의 `GEMINI_MODEL` → `npx wrangler deploy`. 서버는 503 은 다시 시도하고, 429 는 바로 멈춰 Google 의 제한 내용을 보여 줘요. Gemini 가 45초 안에 답하지 않으면 끊어요.

판정 검증 (기획서 8장 다시 하기)

부록 A 그림 4장 + 부록 B 프롬프트 원문(`Server/judge/appendix_b.json`)으로 그림마다 한 번씩 판정 → 2026-10-27 Gemini 로 4장 모두 기획서 표와 같은 결과. 입력 그림과 결과 : `Docs/기록/9_기획서8장_판정_재검증.pdf`. ※ 기획서 8장 돗자리는 십자 네 칸, 게임 3단계 돗자리는 최종 아트대로 위에서부터 색 띠 네 줄이라 프롬프트가 달라요. 게임 1단계 방망이도 최종 아트대로 돌기가 있는 머리 기준이에요.

> 자동 플레이 테스트(MvpSelfTest)는 AI 를 부르지 않아요. 화면 흐름을 보려고 판정 결과를 미리 정해서 넣고, 그 결과에 맞는 손그림(HandSketch)을 제출해요.

5. 화면 만들기 (MvpSceneBuilder)

좌표	`PlaceM(rt, x0,y0,x1,y1)` = 시안 픽셀 상자 그대로 (K = 1080/2248). `TeamImage` = 오린 그림을 원래 자리에, `TeamTool` = 시안 그림 버튼 + 고른 표시 고리
화면 크기	CanvasScaler 1080×1899 Expand, 가운데 `Frame` 안에 모든 화면. 폰이 더 길면 위아래에 방 배경
그림 설정	시안 UI · 시작 화면은 압축 없이 + 밉맵 + 최대 4096 (외곽선이 또렷하게). 시안 부품은 시안 원본 크기예요. 카드 · 버튼은 9-slice (Common_Card · Common_ButtonCoral · Common_ButtonCream)
글꼴	Jua SDF (`MvpFontBake` 가 게임 글자만 구움). 글자를 새로 넣었으면 글꼴 굽기를 다시
단계 데이터	`CreateStageDataFiles` : 이야기 · 규칙 · 특징(이름 · 기준 글) · 그림 연결. 씬 만들 때마다 덮어써요

6. 도구 (Tools → 찾아라도깨비, 또는 Temp/ 의 flag 파일)

도구	파일	하는 일
MVP 씬 만들기	MvpSceneBuilder · MvpAutoBuild	씬을 처음부터 만들기 (mvp_rebuild.flag). 바뀐 코드를 먼저 읽어 들인 뒤 만들어요
자동 플레이 테스트	MvpSelfTest · MvpLayoutCheck	5단계 흐름 · 규칙 · 버튼 · 펜(그림판 밖) 확인 + 화면 캡처(Temp/SelfTest) + 배치 숫자 검사(걸쳐 겹침 · 글자 넘침 · 화면 밖) (mvp_selftest.flag)
게임 플로우 사진 찍기	MvpFlowShots	AI 없이 경우마다 화면을 멈춘 뒤 찍어요 (Temp/FlowShots, mvp_flowshots.flag). 문서는 `Tools/docs/make_flow_doc.py`
판정 전수 검사	MvpJudgeCases · HandSketch	특징 조합 62가지(방망이 굵은 펜 포함 64번) 손그림을 계산식 판정에 넣어 보기 (mvp_judgecases.flag)
글꼴 굽기	MvpFontBake	Jua SDF (mvp_font.flag)
웹 빌드	MvpWebGLBuild	Builds/WebGL + 판정 서버 프롬프트 내보내기 (mvp_webgl.flag)
조합 그림 연결	MvpSceneBuilder · MvpAutoBuild	단계 폴더의 물건_Wrong_*.png 를 단계 데이터에 연결, 씬은 그대로 (mvp_wrongart.flag)
판정 서버 프롬프트	MvpJudgeServer	stages.json 내보내기, server_url.txt → 씬의 서버 주소
몇 단계부터 시작	GameManager	Inspector 의 Editor Start Stage (에디터 전용, 앞 단계는 찾은 것으로 기록). 웹 빌드는 항상 1단계부터
플레이 영상	MvpPlayRecorder	자동 플레이를 프레임으로 저장. 시연 영상은 Unity Recorder 로 찍은 `Recordings/시연용영상.mp4` (git 에 안 올림)

7. 자주 하는 수정

하고 싶은 것	방법
엉뚱한 물건 · 캐릭터를 바꿔요	`Assets/Sprites/Stage*/물건_Wrong_*.png` 저장 → mvp_wrongart.flag → 웹 빌드. 캐릭터는 `Assets/Sprites/Characters` 의 같은 이름으로 덮어쓰기 → 웹 빌드
아트팀 그림 ZIP 이 왔어요	`python Tools/docs/apply_art_zip.py 받은파일.zip --apply` → 자동 테스트 → 웹 빌드 (`Docs/전달/4_아트팀_전달사항.pdf`)
소리 파일이 왔어요	`Assets/Resources/Sound` 에 넣기 (`Docs/전달/12_사운드팀_전달사항.pdf`)
대사를 고치고 싶어요	`GameScript.cs` → 새 글자가 있으면 글꼴 굽기 → `python Tools/docs/make_dialogue_tree.py`
배치를 옮기고 싶어요	MvpSceneBuilder 의 PlaceM 좌표(시안 픽셀) → 씬 만들기 → 자동 테스트의 배치 검사가 0 인지
특징 · 판정 기준	CreateStageDataFiles 의 특징 글 → 씬 만들기 → 판정 서버 프롬프트 내보내기 → wrangler deploy (결과 화면의 특징 이름도 바뀌었으면 웹 빌드도)

판정 기준을 바꿨어요	판정 서버 배포와 웹 빌드를 함께 (기준 수가 달라지면 예전 웹 게임과 서버가 맞지 않아요)
웹에 올리기	웹 빌드 → Builds/WebGL/Build 를 gh-pages 브랜치에 커밋 · 푸시 (GitHub Pages)